

Gigafizz part 1: fizz buzz without `if`

CS 551: Systems Programming

Contents

Introduction	1
Getting started	1
What to do	2
Checks to run before you submit	3
Grading	4
Academic honesty	4
Use the toolchain	4

Prerequisite lectures. Lecture 0 (getting started) and Lecture 2 (enums, pattern matching, control flow).

Due. Friday 28 August 2026, 11:59PM.

Read `README.md` first. The standing rules there apply here, and this assignment does not repeat them. Read `ERRATA.md` before you submit.

Introduction

Read the entirety of this document before you start. Deviations from it lose points.

If you have a question about this document, or you think something in it is ambiguous or wrong, ask. Quote the text you are asking about. Clarifications go in `ERRATA.md`.

Getting started

Accept the Classroom 50 assignment:

<https://classroom50.org/2026-Fall-CS-551/cs-551/assignments/gigafizz/accept?k=zl5swwh0>

You get a working workspace: it builds, it runs, its tests pass, and `cargo clippy` is quiet. It also ships a `HONESTY.md` waiting for your name, and a `README.md` that Step 5 tells you to replace. This is the only repository you accept for gigafizz; every later part adds to it.

Before you write anything, read the whole thing. In particular:

- The two `Cargo.toml` files and the workspace root. Note how dependencies are declared once at the workspace level and inherited.
- `cli/src/main.rs`. Note how subcommands are declared and dispatched.

- `cli/src/commands/simple.rs`. Note how an option gets a long form, a short form, an environment variable, and a default value, and note that the doc comment on the field becomes the help text.
- `core/src/lib.rs`. Note the shape of the documentation on `simple_fizzbuzz`, including the `# Examples` block, and note that the block is a real test.

Confirm it works:

```
cargo run -- simple --on 15
cargo run -- simple --on 7
SIMPLE_ON=9 cargo run -- simple
cargo test
```

What to do

`simple_fizzbuzz` in `core` decides between four cases using a chain of `if` and `else if`. You are going to write a second implementation that reaches the same answers without using `if` at all.

Step 1: take the listings

Replace the `use` line at the top of `cli/src/commands/simple.rs` with:

```
use gigafizz_core::{simple_fizzbuzz, simple_fizzbuzz_no_if_statements};
```

Replace `SimpleOptions` with Listing 1:

```
// Listing 1
#[derive(Args, Debug)]
pub(crate) struct SimpleOptions {
    /// What number to calculate fizz buzz on.
    #[clap(
        long,
        short,
        env = "SIMPLE_ON",
        default_value = "100",
        allow_negative_numbers(true)
    )]
    on: i64,

    /// Use the "no if statements" implementation.
    #[clap(long, short)]
    no_if_statements: bool,
}
```

Replace `simple` with Listing 2:

```
// Listing 2
pub(crate) fn simple(opts: &SimpleOptions) {
    tracing::debug!(?opts, "running simple");
    if opts.no_if_statements {
        println!(
            "simple_fizzbuzz_no_if_statements({}): {}",
            opts.on,
            simple_fizzbuzz_no_if_statements(opts.on)
        );
    } else {
```

```

        println!("simple_fizzbuzz({}): {}", opts.on, simple_fizzbuzz(opts.on));
    }
}

```

These three edits are the only changes you may make under `cli/`. The project does not compile after you make them. That is expected: you have just written down a call to a function that does not exist yet.

Step 2: make it compile

Write the missing function in `core/src/lib.rs`. It must have exactly this signature:

```
pub fn simple_fizzbuzz_no_if_statements(input: i64) -> String
```

It returns the same answers `simple_fizzbuzz` does:

Condition	Returns
input divisible by 3 and by 5	"FizzBuzz"
input divisible by 3 only	"Fizz"
input divisible by 5 only	"Buzz"
neither	the number, as a string

Step 3: obey the constraint

The body of your function must not contain any if of any kind. No `if`, no `else if`, no `if let`, no `else`. Not in a helper function you wrote for it either.

This is the entire point of the part. Rust gives you a construct that expresses a four-way decision better than a chain of `ifs` does, and this constraint exists to make you find it.

Step 4: document it

Your function carries a rustdoc comment that includes an `# Examples` section. The examples run as doctests, so they have to compile and pass. `cargo test` runs them.

Use the documentation on `simple_fizzbuzz` as your model.

Step 5: your README.md

Add a `README.md` at the root of your repository containing:

1. A short description of what you did.
2. The exact output of `rustc --version` on the machine you developed on.
3. Any crate you added, and why. (You should not need to add any.)

Checks to run before you submit

```

cargo clippy --all-targets  # fix what it reports
cargo fmt                  # then confirm `cargo fmt --check` is silent
cargo test
cargo doc --no-deps --open

```

`cargo clippy` may tell you something about where in the file you put your function. It is right. Fix it rather than silencing it.

Grading

Out of 100.

Zero on this part if any of these are true:

- Anything under `cli/` is modified beyond the three edits in Step 1.
- The workspace does not compile.
- The binary is not named `gigafizz`.
- Your repository has no `HONESTY.md` containing the academic honesty statement from the syllabus.
- `target/` is committed to the repository.

Otherwise:

Points	Item
40	<code>--no-if-statements</code> produces correct output. Ten points each for the four cases in the table.
15	The function has exactly the required name and signature.
15	No <code>if</code> of any kind appears in your implementation.
10	Rustdoc with an <code># Examples</code> section that passes as a doctest.
10	<code>cargo clippy --all-targets</code> is silent, with no <code>#[allow(...)]</code> used to silence anything.
5	<code>cargo fmt</code> produces no diff.
5	<code>README.md</code> contains all three required items.

Academic honesty

Your repository must contain a `HONESTY.md` with the academic honesty statement from the syllabus, followed by your name.

This course does not permit the use of generative AI tools, for code, for documentation, or for correspondence. See the syllabus.

Use the toolchain

The prohibition is on generative AI, not on tooling. **You are encouraged to use the Rust toolchain as fully as you can.** Cargo does a great deal more than `build`, `test`, and `run`, and finding out what it does belongs in this course:

- `cargo clippy --fix` applies the suggestions it can apply mechanically, and implies `--no-deps` and `--all-targets`. It is not a button that empties the warning list: only suggestions marked machine-applicable get rewritten, some lints have no automatic fix at all, and clearing one can uncover another underneath it. Run it, read what it changed, then finish the list by hand. It also refuses to run with uncommitted changes in the tree, which is good advice rather than an obstacle: commit first, so you can see what it did.

- `cargo doc --no-deps --open` renders your documentation the way a reader meets it, which is the fastest way to notice a public item you never documented.
- `cargo tree` shows what your dependencies dragged in behind them.
- `cargo add` edits `Cargo.toml` for you, and records the version it picked.

Ask before you rely on a cargo extension. The subcommands you install with `cargo install cargo-<something>` are interesting and mostly welcome, but name the one you want and say what you want it for. A tool that writes the code for you is still prohibited however it is packaged.